

## Technical Report

Subject: Lane Departure Warning System, DP01  
Date: 05/04/2026



### Technical Report Identification Table

| Item           | Description                                                          |
|----------------|----------------------------------------------------------------------|
| Report Title   | Lane Departure Warning System, Delivery Phase 01                     |
| Author/Authors | Asal Sanei, Amirhossein Mohammadi, Kourosh Karimi, Fatemeh Javaherat |
| Reviewer       | Mohammadreza Maleki                                                  |
| Final approver | Hamidreza Rezaei                                                     |
| Version        | 1.0                                                                  |
| Issue Date     | 05/17/2026                                                           |
| Status         | Final                                                                |

## Executive Summary

This report presents the design and implementation of a Vision-Based Lane Departure Warning (VBLDW) system using classical computer vision techniques. The primary objective of the project was to develop a lightweight and interpretable lane detection framework capable of identifying lane boundaries and warning drivers when unintended lane departure occurs under both highway and urban driving conditions.

The proposed system processes video frames through a sequence of image processing stages, including grayscale conversion, Gaussian filtering, Canny edge detection, Region of Interest (ROI) masking, Hough Transform, and lane line selection. Unlike modern deep learning approaches, the implemented method does not rely on neural networks or extensive computational resources, making it suitable for low-cost and real-time embedded applications.

To evaluate the robustness of the pipeline, multiple driving scenarios were considered, including clear highway environments, tunnel conditions, dashed lane markings, and challenging urban scenes with shadows and faded lanes. In addition, several noise models, including Gaussian, Salt-and-Pepper, and Speckle noise, were introduced to simulate real-world sensor imperfections and environmental disturbances. Ground truth annotations were manually generated using MATLAB Ground Truth Labeler to support frame-by-frame evaluation.

The warning mechanism was implemented using the Lateral Offset Ratio (LOR), a calibration-free metric that estimates the vehicle's lateral position relative to lane boundaries. Based on the calculated LOR values, the system classifies the driving condition into safe driving, warning threshold, or lane departure states. Experimental results demonstrated that the proposed workflow successfully detects lane boundaries and identifies lane departure events while providing visual warnings to the driver.

Overall, the project demonstrates that traditional computer vision methods remain effective for lane departure warning applications, particularly in scenarios where computational efficiency, simplicity, and explainability are important design considerations.

## 1. Scope & Objectives

This report covers the implementation of a classic computer vision pipeline for lane detection, designed to operate on real-world video footage under highway and urban driving conditions. The system processes video frames using traditional image processing techniques (ROI, Cropping, Canny Edge Detection, Hough Transform) without employing any machine learning or deep learning models.

## 2. Data Pipeline

The data pipeline covers all activities related to video collection, noise injection, and ground truth label generation. It provides the foundation for evaluating the lane detection system against the acceptance criteria of the project. Figure 2.1 indicates the relevant data area.

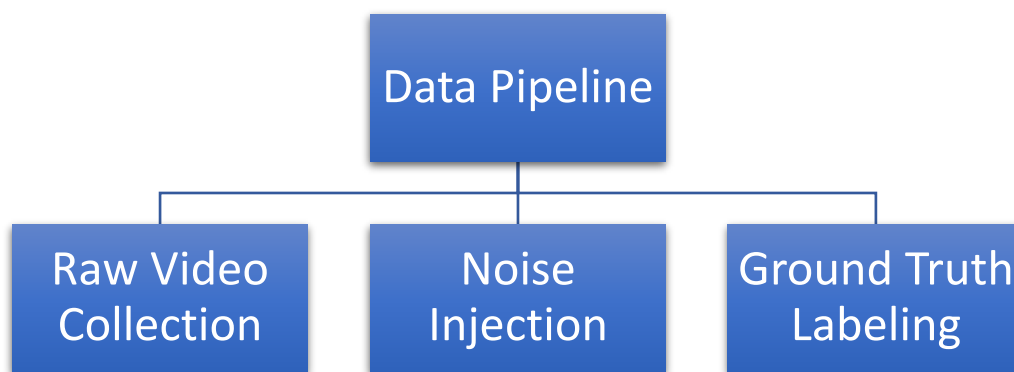


Figure 2.1. Data Pipeline Relevant Tasks

### 2.1. Raw Video Files

Raw video files are collected from two sources: self-captured footage using a smartphone from a moving vehicle and downloaded videos from public sources. Two contrasting conditions are represented in the following.

- Condition A: Clear highway with well-marked lanes downloaded and selected from a GitHub repository and Google Drive [1] [2].
- Condition B: Urban environment with shadows, occlusions, and faded lane markings recorded using a smartphone.

Videos are named consistently to encode condition, source, and sequence number. The downloaded dataset is a multi-lane detection dataset, which can be used to test and evaluate multiple lane detection algorithms. It is collected by a road-facing traffic recorder in some sections of Ji Qing (Jinan-Qingdao) expressway, China. Total 3 videos are selected and cropped

from this dataset to match the Euro NCAP 2023 Test Protocol for Lane Support Systems as described in the following [3].

- Dashed Left Line Test Scenario present intermitted edge segments on the left side and the vehicle lane changes from left as shown in Figure 2.2.



Figure 2.2. Dashed Left Line Test Scenario

- Dashed Right Line Test Scenario as illustrated in Figure 2.3, presents the same challenge on the right side, ensuring symmetric performance.



Figure 2.3. Dashed Right Line Test Scenario

- Solid Lines Test Scenario provides continuous strong edges and is recorded in a tunnel as show in Figure 2.4.



Figure 2.4. Solid Lines Test Scenario

The frame rate of these videos is reduced to 4 [fps] to output 60 frames for each of them due to simplicity and project proposal.

A single 4 [fps] video is also self-captured in an urban environment satisfying Condition B using a smartphone from a passenger car as shown in Figure 2.5.



Figure 2.5. Self-Captured Video

## 2.2. Noise Injection

Noise injections are applied to evaluate system robustness. Three distinct noise types are implemented as follows.

- Gaussian noise, which simulates sensor thermal variation and appears as a subtle grainy texture across the entire image, is an additive noise that typically disrupts the gray values in digital images. For this reason, the Gaussian noise model is essentially

defined by the probability density function in terms of gray value as expressed in Equation 2.1 or by normalizing the histogram in terms of gray value.

$$p(g) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(g-\mu)^2}{2\sigma^2}} \quad (\text{Equation 2.1})$$

Where  $g$  is the gray value,  $\sigma$  is the standard deviation and  $\mu$  is the mean value. Figure 2.6 shows a Gaussian noised frame with  $\mu = 0$  and  $\sigma = 0.01$ .



Figure 2.6. Gaussian Noised Frame

- Salt-and-Pepper noise, which simulates dead pixels or bit errors and appears as random white and black dots, is not additive unlike gaussian noise. Instead of adding a random value to every pixel, it randomly places a percentage of pixels with either the minimum or maximum possible intensity value. Salt noise refers to the corrupted pixels that take the absolute maximum value (e.g., 255 in an 8-bit grayscale image), appearing as stark white dots. On the other hand, pepper noise leads to corrupted pixels that take the absolute minimum value (e.g., zero in an 8-bit grayscale image), appearing as stark black dots. A sample salt and pepper noised frame with *density* = 0.05 is shown in Figure 2.7.





Figure 2.7. Salt and Pepper Noised Frame

- Speckle noise are complex noises because their nature is multiplicative rather than additive. In fact, they are multiplied by the signal so that if they are zero, they destroy the signal as demonstrated in Figure 2.8.



Figure 2.8. Speckle Noised Frame

Noise is added to individual frames in memory rather than being written to disk. This decision avoids unnecessary storage overhead, keeps the Git repository free of large binary files, and ensures that the original clean videos remain available for baseline comparison.

### 2.3. Ground Truth Data and Labeling

There are several data structures for composing ground truth data. While JSON is a great universal format, using MATLAB's built-in labeling tool seems better for this project requirements. The native supported object notation is the *.mat* format that can be created using MATLAB Ground Truth Labeler App.

Ground truth data is created through manual annotation. For each of the two main conditions, 60 frames are selected from the corresponding video. These frames are loaded into the MATLAB Ground Truth Labeler application, where a human annotator draws polyline

boundaries along the left and right lane markings. Each line is represented by 2 points indicating start and end of the line. Figure 2.9 shows an example of a ground truth frame labeling.



Figure 2.9. Ground Truth Labeling

The annotations are exported as a single *.mat* file per video, containing an array of 60 frame records. Each record includes the frame identifier and the coordinates of all lane boundaries. This structured format allows the evaluation script to compare the pipeline's output against the ground truth on a frame-by-frame basis using the Intersection over Union metric (IOU).

### 3. Lane Detection

The lane detection pipeline converts each frame to grayscale, applies Gaussian smoothing and Canny edge detection to obtain a binary edge map, then restricts processing to a trapezoidal region of interest. Hough transform with peak detection extracts candidate lines. Finally, the left and right lines are chosen based the implemented algorithm in a specific function designed for line selection. A n overview of the implemented process is shown in Figure 3.1.

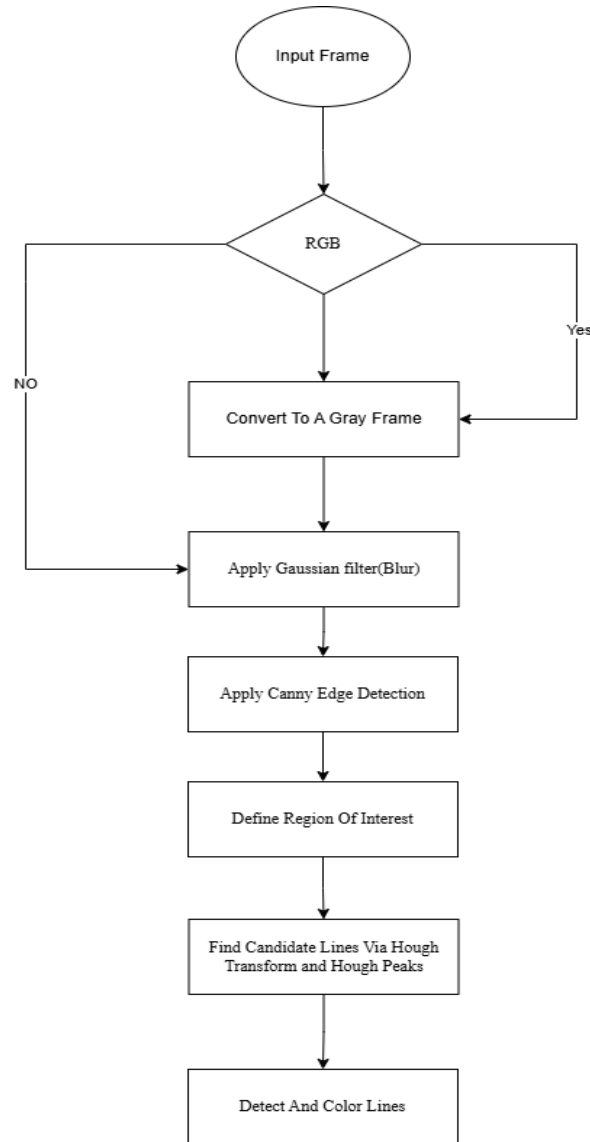


Figure 3.2 Lane Detection Process Flowchart

## 3.1. Preprocessing

First, the input is converted to a grayscale image. This is because the subsequent edge detection step (Canny) fundamentally operates on single-channel intensity gradients, not on color.

Secondly, a slight Gaussian filter ( $\sigma = 0.2$ ) is applied to suppress noise while preserving edge sharpness. Like any other filter, the gaussian filter applies weighted average to neighboring pixels in the image except that here the kernel weights are based on gaussian distribution as given in Equation 3.1.

$$G(x, y) = \left(1/(2\pi\sigma^2)\right) \cdot e^{-(x^2+y^2)/(2\sigma^2)} \quad (\text{Equation 3.1})$$



The Gaussian Function gives higher weight to the pixels closer to the center of the kernel because of the exponential decay. This helps to preserve the edges compared to Uniform averaging like in a box filter. The visualization of this filter is represented in Figure 3.2.

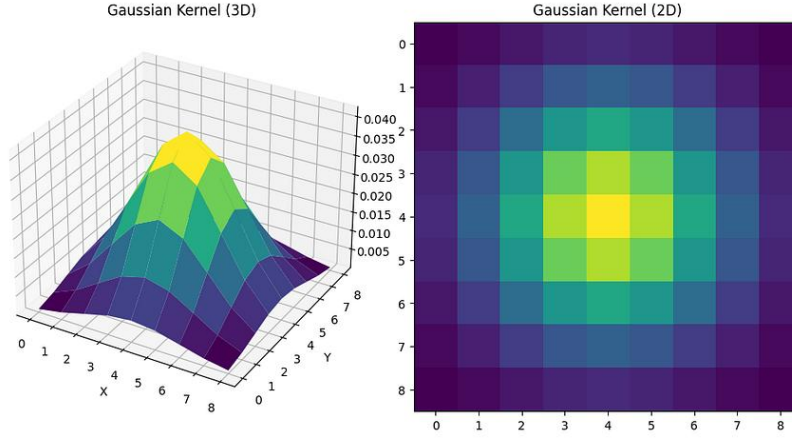


Figure 3.3: Gaussian 3D and 2D Kernels [4]

Effect of Sigma ( $\sigma$ ): The choice of  $\sigma$  represents a fundamental tradeoff [5]:

- Small  $\sigma$  (0.5–1): preserves fine details and precise edge locations, but leaves more noise untouched. This is useful for images with subtle features and minimal noise.
- Large  $\sigma$  (>2) aggressively removes noise but also blurs edges, potentially merging close edges together and losing fine details. This works better for noisy images where edge precision is less critical.

### 3.1.1. Canny Edge Detection

Canny edge detector operates through a sequence of mathematically grounded stages designed to meet three key criteria: good detection (maximizing signal-to-noise ratio), good localization, and a single response to each edge.

After implementing the Gaussian Filter, gradient computation identifies locations of rapid intensity change. Sobel operators (3×3 convolution masks) approximate the horizontal and vertical derivatives  $G_x$  and  $G_y$ . From these components, the gradient magnitude  $M$  (edge strength) and direction  $\theta$  (edge orientation) are computed as below in Equation 3.2 and Equation 3.3:

$$M = \sqrt{(G_x^2 + G_y^2)} \quad \text{(Equation 3.2)}$$

$$\theta = \text{atan2}(G_y, G_x) \quad \text{(Equation 3.3)}$$

The magnitude indicates edge strength, while the direction is perpendicular to the edge itself.

Non-maximum suppression (NMS) refines the gradient magnitude map to one-pixel-wide edge contours by performing a local maximum test along the edge direction. The gradient orientation  $\theta$  is first quantized to one of four canonical directions: 0°, 45°, 90°, or 135°, each corresponding to the direction of the edge normal (perpendicular to the gradient). For each

pixel, the algorithm compares its magnitude  $M(x,y)$  with the two immediate neighbors along that quantized edge direction. If  $M(x,y)$  is strictly greater than both neighbor magnitudes, the pixel is retained as a candidate edge point; otherwise, it is suppressed (set to zero). This ensures that only the maxima of the gradient magnitude ridge are preserved, yielding edges that are both thin and accurately localized. [5]

### 3.1.2. Region Of Interest

Region of Interest (ROI) refers to a deliberately chosen subset of an image that is processed further, while the rest of the image is ignored. The purpose is to focus computational resources on the portion that actually contains the information of interest, thus improving both speed and robustness. In this project a trapezoidal mask is defined that discards image regions unlikely to contain lane markings (sky, neighboring vehicles, far periphery). The mask's bottom corners are placed at 20 % and 80 % of the image width, and the top corners at 35 % and 65 % of the width, while the top edge sits at 55 % of the image height. Only edge points inside this polygon are retained.

### 3.1.3. Hough Transform And Hough Peaks

Hough Transform which is a popular line detection algorithm that works by mapping points in an image to lines in parameter space. The Hough transform can detect lines of any orientation and can work well in images with a large amount of noise. In order to implement this algorithm, a line should be described by  $\rho$  the perpendicular distance from the origin and  $\theta$  the angle made by the perpendicular with the axis as shown in Figure 3.3:

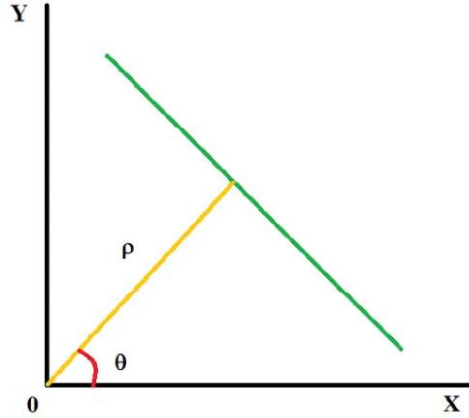


Figure 3.4: Line representation [6]

The equation of the line is therefore given as Equation 3.4:

$$y = -(\cos \theta / \sin \theta) x + (\rho / \sin \theta) \quad (\text{Equation 3.4})$$

By rearranging the Equation 3.4, the line is obtained as:

$$\rho = x \cos \theta + y \sin \theta \quad (\text{Equation 3.5})$$

From Equation 3.5, it is implied that all the points having the same values of  $\rho$  and  $\theta$  constitute a single line.

Hough Transform is implemented for  $\theta$  limited to interval  $[-78^\circ, 78^\circ]$  which excludes lines that are near-horizontal.

Finally, Hough Peak function is used. This function extracts the most salient linear structures from the Hough accumulator array  $H$ . It operates by first identifying cells that represent strict local maxima within their 8-connected neighborhood, ensuring that only the central point of each accumulator cluster is selected. A relative threshold, defined as 30 % of the global maximum accumulator value (rounded up), then prunes candidates that lack sufficient support, thereby suppressing spurious alignments caused by noise or short edge fragments. From the remaining set, at most the ten highest-valued peaks are retained, and their row and column indices are subsequently mapped to physical line parameters  $(\rho, \theta)$  via the parameter vectors output by Hough, providing a compact set of dominant line hypotheses for the subsequent lane-selection stage.

### 3.1.4. Selecting Lines

The lane candidate selection proceeds by first partitioning the detected Hough peaks into left ( $\theta < 0$ ) and right ( $\theta > 0$ ) groups.

For each group, the mean of all  $\rho$  values and the mean of all  $\theta$  values are computed via a function. These averaged parameters then define the final left and right lane lines. This averaging approach is robust against noise and fragmented line detections (e.g., dashed lane markings) because it combines evidence from all candidate segments instead of relying on a single peak.

## 4. Warning Logic: Lateral Offset Ratio

The Lateral Offset Ratio (LOR) is a dimensionless computational metric utilized within Vision-Based Lane Departure Warning (VBLDW) frameworks to predict unintended vehicle drift [7]. In modern automotive engineering, lateral offset is defined as the distance between a vehicle's centerline and the lane's centerline, serving as a primary variable for assessing driving performance and car-following efficacy. A primary academic and practical advantage of the LOR method is that it operates without the need for intrinsic or extrinsic camera calibration, making it significantly more flexible and cost-effective than traditional techniques requiring precise camera parameters[8].

### 4.1. Mathematical Specification

The LOR is calculated for every video frame using the horizontal coordinates of the bottom end-points of detected lane boundaries within the image plane.

The governing equation for LOR is[7]:

$$LOR = \frac{\min\{|X_{12} - X_m|, |X_{22} - X_m|\} - T_H X_m}{T_H X_m} \quad (\text{Equation 4.1})$$

## Variable Definitions:

- $X_{12}$  : The detected left bottom end-point of the left lane boundary.
- $X_{22}$  : The detected right bottom end-point of the right lane boundary.
- $X_m$  : One-half of the horizontal width of the image plane, representing the camera's optical center line.
- $T_H$  : The lane departure warning threshold, defined as a constant value of 0.8 [9].

This ratio represents the relationship between the horizontal pixel distance from the warning threshold to the detected lane boundary and the distance from the threshold to the center line. The visualized of this parameter based on the image could be found in bellow:

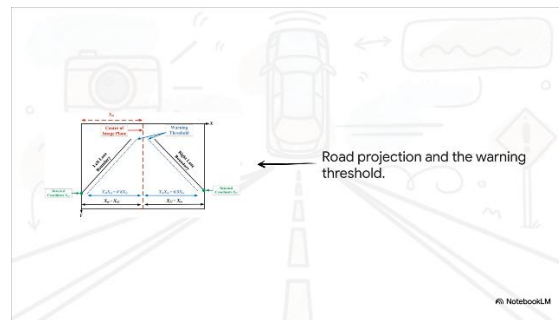


Figure 4.1 Lane detected parameters on image

## 4.2. Classification of Lane Departure Events

The Classification of Lane Departure Events is primarily determined by the specific value of the Lateral Offset Ratio (LOR), which categorizes the vehicle's state into operational contexts ranging from centered driving to a full boundary crossing. A calculated LOR of 0.25 indicates the vehicle is perfectly centered within its lane, while values between 0 and 0.25 represent normal driving within a "safe zone" where no warning is triggered. The system identifies a critical warning limit when the LOR reaches 0, signifying that the vehicle has exactly crossed the predefined warning threshold—often set at 80% of the lane width per ISO 17361:2007 standards[7]. Values between -1 and 0 place the vehicle in the departure zone, indicating it is drifting into an adjacent lane, and a minimum LOR value of -1 identifies a complete boundary crossing where the vehicle is physically intersecting the lane boundary.

| LOR Value                  | Lane Departure Status |
|----------------------------|-----------------------|
| LOR = 0.25                 | No departure          |
| $0 < \text{LOR} \leq 0.25$ | No departure          |
| LOR = 0                    | Departure Warning     |
| $-1 \leq \text{LOR} < 0$   | Departure Warning     |

Figure 4.2 LOR classification value

## Result

After developing all the components described above, we evaluated the system using two test frames: one representing a normal driving condition within the lane and another representing a lane departure scenario. The results demonstrated that the proposed workflow operates correctly and is capable of detecting lane departures while providing an appropriate visual warning.

As illustrated in Figure 5.1, the vehicle remains within the lane boundaries, and therefore the Lateral Offset Ratio (LOR) is expected to be positive. The image processing algorithm successfully detects the lane markings, while the warning logic based on the LOR accurately determines the vehicle's lane status and identifies potential departures.

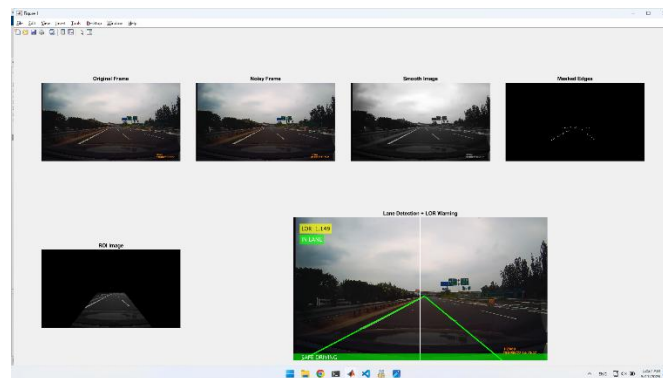


Figure 5.1 positive LOR and safe driving

In another test scenario, the system was evaluated using a frame in which the vehicle was crossing the lane boundary. In this situation, both visual and auditory warnings should be activated to alert the driver. As shown in Figure 5.2, the classical image processing algorithm successfully detected the lane markings, and the warning logic accurately identified the lane departure event. At this stage, a red visual warning is displayed on the image to indicate that a lane departure has occurred.

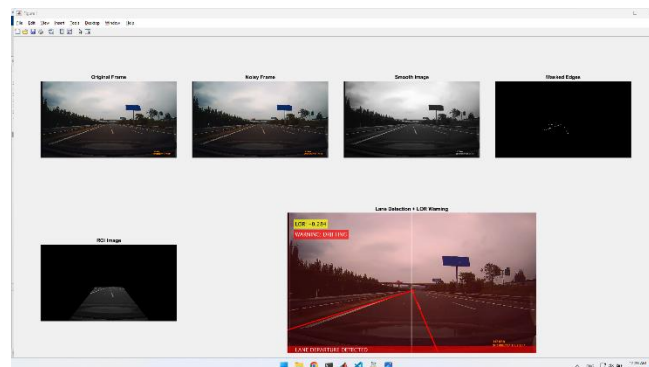


Figure 5.2 Negative LOR and the visual warning



## Conclusion

This project successfully implemented a classical Vision-Based Lane Departure Warning system capable of detecting lane boundaries and identifying lane departure events using conventional image processing techniques. The developed pipeline integrates preprocessing, edge detection, region masking, Hough Transform-based line extraction, and warning logic based on the Lateral Offset Ratio (LOR) to create a complete lane monitoring framework.

One of the major advantages of the implemented system is its simplicity and low computational complexity compared to deep learning-based alternatives. Since the proposed method does not require camera calibration or large-scale model training, it offers a practical and cost-effective solution for lightweight Advanced Driver Assistance Systems (ADAS) and real-time embedded platforms.

Despite these advantages, the system still faces challenges in highly complex environments, such as severe shadows, occlusions, worn lane markings, and rapidly changing lighting conditions. Future improvements may include adaptive thresholding, temporal lane tracking, sensor fusion, or hybrid approaches that combine classical computer vision with machine learning techniques to improve robustness and accuracy.

At this stage, the proposed algorithms have been evaluated only on a limited number of frames. For a more comprehensive assessment, future testing should be conducted on larger datasets containing a wider variety of frames and driving scenarios. In addition, quantitative evaluation metrics such as Intersection over Union (IoU) should be employed to measure detection performance more accurately. Further optimization of the internal algorithm parameters is also necessary to improve the overall robustness and accuracy of the system.

In conclusion, the project demonstrates that traditional image processing algorithms can provide an effective foundation for lane departure warning systems and can serve as a strong baseline for future ADAS research and development.

## References

- [1] "Multi-Lane-Detection-Dataset-with-Ground-Truth, GitHub Repository," [Online]. Available: <https://github.com/vonsj0210/Multi-Lane-Detection-Dataset-with-Ground-Truth>. [Accessed May 2026].
- [2] "Lane Dataset - Google Drive," [Online]. Available: [https://drive.google.com/drive/folders/1iO6EUira1\\_irMHnEFjma3vc8LfYKHmQ4](https://drive.google.com/drive/folders/1iO6EUira1_irMHnEFjma3vc8LfYKHmQ4). [Accessed May 2026].
- [3] "Euro NCAP LSS Test Protocol v4.2," [Online]. Available: [https://cdn.euroncap.com/cars/assets/euro\\_ncap\\_lss\\_test\\_protocol\\_v42\\_bfc543c53f.pdf](https://cdn.euroncap.com/cars/assets/euro_ncap_lss_test_protocol_v42_bfc543c53f.pdf). [Accessed May 2026].
- [4] V. Vignesh, "towardsai," 17 July 2023. [Online]. Available: <https://towardsai.net/p/l/gaussian-blurring-a-gentle-introduction>.
- [5] Z. Ansari, "Medium," 11 May 2025. [Online]. Available: [https://medium.com/@zar\\_373/the-canny-edge-detector-mathematical-foundations-and-implementation-of-a-computer-vision-c47e83d6e792](https://medium.com/@zar_373/the-canny-edge-detector-mathematical-foundations-and-implementation-of-a-computer-vision-c47e83d6e792).

- [6] Shantanuparab, "Medium," 23 May 2023. [Online]. Available: <https://medium.com/@shantanuparab99/hough-transform-3e5f6875b9b8>.
- [7] J. Wu, P. Yin, X. Shu, H. Huang, F. Liu and Q. Wu, "A Vision-based Lane Departure Warning Framework," *2021 IEEE International Conference on e-Business Engineering (ICEBE)*, pp. pp. 139-143, 2021.
- [8] E. P. Ping, "Data fusion based lane departure warning framework using," Malaysia, 2022.
- [9] *Intelligent transport systems — Lane departure warning systems — Performance requirements and test procedures ISO17361*.

## Appendices